

System 14: Distributed Web Crawler (Google / Bing Scale)

Preceding Bridge: In [System 12: Distributed Search Engine \(Elasticsearch\)](#) and [System 04: Distributed Task Scheduler](#), you learned inverted indexing and priority scheduling. In this chapter, we design the global ingestion pipeline that powers search engines: a **Distributed Web Crawler** capable of fetching billions of web pages monthly with strict domain politeness, duplicate detection, and crawl-trap defenses.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
Seed URLs	The initial list of well-known starting web addresses given to the crawler.	The port from which a fleet of exploratory ships begins their voyage.
URL Frontier	The distributed scheduling queue managing which links to visit next while enforcing priorities and delays.	An airport air traffic controller scheduling runway slots for incoming and outgoing flights.
Politeness Policy	Limiting requests to any single website so the crawler does not accidentally crash the server (DDoS).	Never calling the same friend on the phone more than once every 5 minutes.
Robots.txt	A standardized text file on web servers dictating which paths automated robots are forbidden to crawl.	A "Staff Only / No Trespassing" sign posted on an office door.
SimHash / MinHash	An algorithm that generates a fingerprint of a web page such that nearly identical pages have nearly identical hashes.	Facial recognition that recognizes a person even if they change their glasses or hairstyle.
Crawl Trap	Infinite loops intentionally or unintentionally created by web servers (e.g., dynamic calendar dates <code>?date=2026-10-06</code> , <code>2026-10-07 ...</code>).	A hall of mirrors where an explorer walks endlessly without finding an exit.

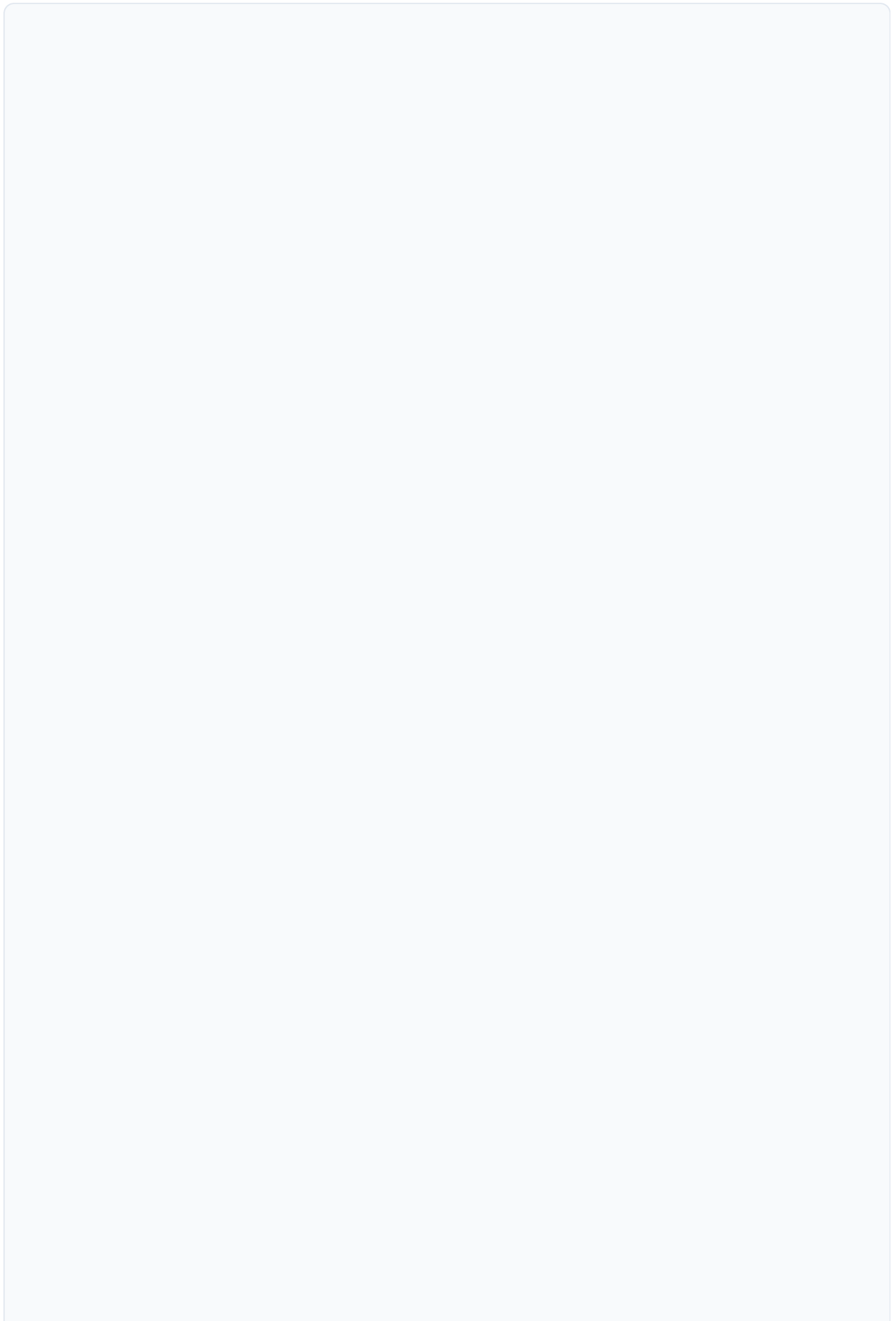
2. Spoon-Fed Mental Model: The Fleet of Deep-Sea Explorers

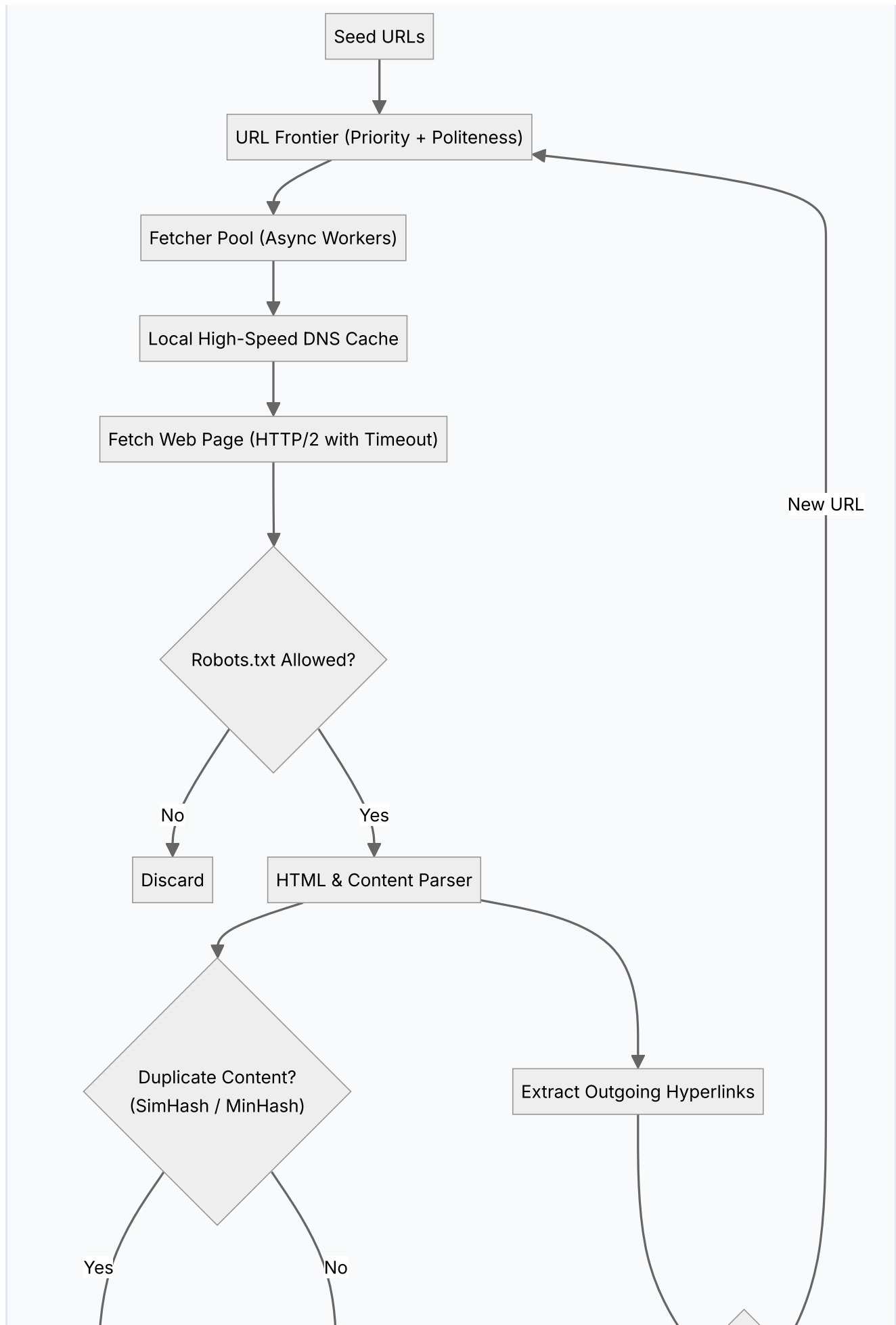
Imagine launching 1,000 deep-sea exploratory drones across the oceans:

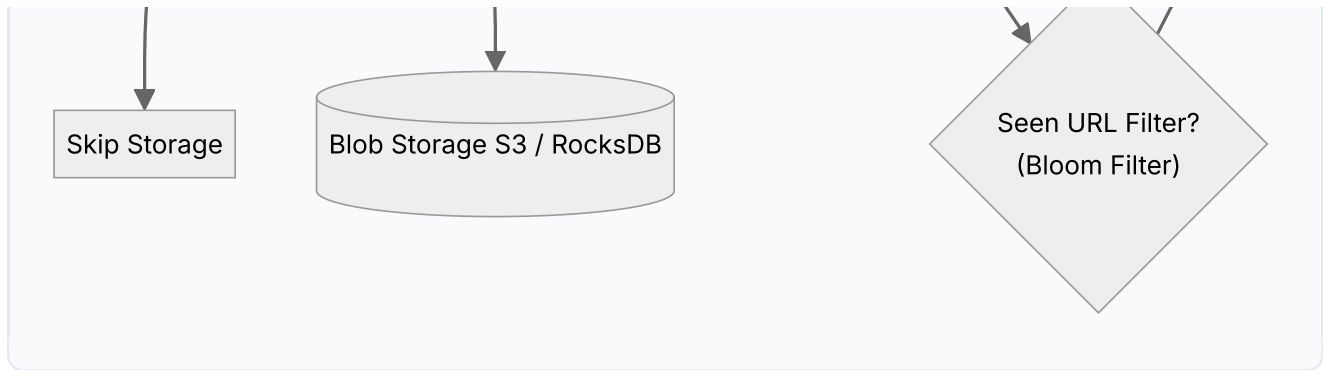
- If all 1,000 drones dive into the exact same coral reef at the same millisecond, they will crush the reef (**Denial of Service**).
- If drones revisit the same shipwreck 500 times because other ships keep mentioning it, fuel is wasted (**Duplicate Fetching**).
- If a drone enters an underwater cave with infinite reflecting mirrors, it gets trapped forever (**Crawl Trap**).

The Master Architecture of a Scaled Crawler:

1. **The Dispatcher (URL Frontier):** Groups URLs by domain name so only **ONE** drone visits `example.com` at any second (Politeness).
2. **The Memory Bank (SimHash Filter):** Before saving a page, checks whether the text is a 99% copy of an existing page.
3. **The Depth Gauge (Trap Defense):** Caps maximum path depth (e.g., max 15 directory levels) and query parameters to avoid infinite calendar traps.







3. Scale & Back-of-the-Envelope Calculations

- **Monthly Ingestion Target:** 1, 000, 000, 000 (1 Billion) web pages.
- **Throughput Required:**

$$\text{Pages per second} = \frac{1,000,000,000}{30 \times 86,400} \approx 385 \text{ pages/sec}$$

Peak multiplier (2×): ~ 800 pages/sec.

- **Storage Footprint:**
 - Average web page raw size (HTML + metadata): 500 KB.
 - Compressed storage per page: ≈ 100 KB.

$$\text{Monthly Storage} = 10^9 \times 100 \text{ KB} = 100 \text{ TB / month} \implies 1.2 \text{ PB / year}$$

- **Network Bandwidth:**

$$\text{Ingress Bandwidth} = 400 \text{ pages/sec} \times 500 \text{ KB} = 200 \text{ MB/sec} = 1.6 \text{ Gbps}$$

4. The URL Frontier Architecture (Mercator Model)

How does Google ensure both **Priority** (crawling high-page-rank news sites first) and **Politeness** (not hammering any single host)?



5. Junior vs Staff Implementation



6. Complete Runnable Implementation: Politeness URL Frontier

Here is a pure Python simulation of an enterprise URL Frontier managing domain politeness with a delay priority heap:



7. Chapter Milestone Check

Verify your understanding before continuing:

1. How does the Mercator URL Frontier architecture satisfy both priority and politeness?

- *Answer:* It uses a two-stage queue system: Front Queues partition URLs by importance/priority, and Back Queues partition URLs strictly by host/domain name. A heap-based politeness scheduler ensures no host's back queue is queried faster than its allowed cooldown window.

2. Why is DNS resolution one of the biggest bottlenecks in a web crawler, and how is it solved?

- *Answer:* Standard synchronous DNS queries take 20–200ms over UDP and can easily choke thousands of worker threads. Crawlers maintain an in-memory high-throughput DNS cache with custom TTLs and use asynchronous DNS resolvers (e.g., `c-ares`).

3. What is the difference between exact duplicate detection and near-duplicate detection?

- *Answer:* Exact duplicates (identical HTML bytes) are caught via MD5/SHA256 hashes or Bloom filters. Near-duplicates (identical news articles with different timestamps or header ads) are detected using locality-sensitive hashing (SimHash or MinHash), where similarity is measured via Hamming distance.